Конвертация в SQL типы

Содержание

1. Проблема 2
2. Решение — Использование `From`, `ToSql` и `FromSql` для преобразования типов. 4
   - 2. 1. Создаём модуль конвертеров 4
   - 2. 2. Преобразуем объект в `Value` 5
   - 2. 3. Передаём объект через `ToSql` 5
   - 2. 4. Читаем объект через `FromSql` 6
   - 2. 5. Восстанавливаем составной объект 6
   - 2. 6. Записываем составной объект 8
   - 2. 7. Что получаем в итоге 8

Задачи 9

1. Проблема

Рассмотрим следующий пример:

У нас есть структура:

```
pub struct Item {
    pub(crate) id: ItemId,
    pub(crate) name: Name,
    pub(crate) price: Price,
    pub(crate) record_lifetime: RecordLifetime,
}
```

Она состоит из следующих объектов-значений:

- `ItemId` — идентификатор элемента.
- `Name` — название элемента.
- `Price` — цена элемента.
- `RecordLifetime` — время жизни записи.

```
pub struct ItemId(pub EntityId);

pub struct Name(String);

pub struct Price(f64);

pub struct RecordLifetime {
    pub(crate) created_at: UtcDateTime,
    pub(crate) updated_at: Option<UtcDateTime>,
    pub(crate) deleted_at: Option<UtcDateTime>,
}
```

Каждый объект-значения — наша собственная структура, которая содержит данные и методы для работы с ними.

Примечание:

Но есть проблема — адаптер с базой данных не может работать с нашими объектами-значениями напрямую.

```
let id: uuid::Uuid = row.get(0)?;
let name: String = row.get(1)?;
let price: f64 = row.get(2)?;

let item_id: EntityId = EntityId::create(id).map_err(|_| {
    rusqlite::Error::ToSqlConversionFailure(Box::new(std::io::Error::new(
        std::io::ErrorKind::Other,
        "Failed to create EntityId",
    )))
})?;

let name: Name = Name::new(name).map_err(|_| {
    rusqlite::Error::ToSqlConversionFailure(Box::new(std::io::Error::new(
```

```
        std::io::ErrorKind::Other,
        "Failed to create Name",
    )))
})?;

let price: Price = Price::new(price).map_err(|_| {
    rusqlite::Error::ToSqlConversionFailure(Box::new(std::io::Error::new(
        std::io::ErrorKind::Other,
        "Failed to create Price",
    )))
})?;
```

Здесь `row` — это строка из ответа базы данных. В `row.get` мы передаем номер столбца (начиная с нуля), значение которого хотим получить.

И каждый раз нам приходиться выполнять преобразование из базового типа в наш объект-значение.

Более того, нам постоянно нужно обрабатывать ошибки при преобразовании.

Это пример, когда у нас трудности с преобразованием данных из базы данных в наши объекты-значения.

Другой пример, это когда мы хотим преобразовать наши объекты-значения в базовые типы для записи в базу данных.

```
async fn add(&self, item: crate::entities::Item) -> StorageResult<()> {
    const QUERY: &str = "
        INSERT INTO items (id, name, price, created_at, updated_at, deleted_at)
        VALUES (?1, ?2, ?3, ?4, ?5, ?6)
    ";

    let parameters: Vec<Value> = vec![
        item.id.into(),
        item.name.into(),
        item.price.into(),
        item.created_at.to_string().into(),
        item.updated_at.map(|dt| dt.to_string()).into(),
        item.deleted_at.map(|dt| dt.to_string()).into(),
    ];

    self.connection
        .conn(move |connection| {
            let affected = connection.execute(
                QUERY,
                params_from_iter(parameters),
            )?;
            Ok(affected)
        })
        .await
        .map_err(|_| DatabaseError::ConnectionError)?;

    Ok(())
```

Здесь вообще ошибка компиляции, потому что мы пытаемся преобразовать следующие структуры в `Value`, который является каким-то базовым типом для SQL:

```
pub struct ItemId(pub EntityId);

pub struct Name(String);

pub struct Price(f64);
```

В базовые типы: `String`, `f64` и другие.

2. Решение — Использование `From`, `ToSql` и `FromSql` для преобразования типов.

Преобразование нужно определить один раз для каждого скалярного объекта-значения, а затем использовать его во всех SQL-запросах. Для этого понадобятся три разных трейта:

- `From<Объект> for Value` создаёт принадлежащее нам SQL-значение и позволяет вызывать `.into()`.
- `ToSql` позволяет передавать объект или ссылку на него в параметры запроса.
- `FromSql` позволяет читать объект непосредственно через `row.get()`.

`Value` — это перечисление представлений SQLite: `Null`, `Integer`, `Real`, `Text` и `Blob`, а не один базовый Rust-тип.

2. 1. Создаём модуль конвертеров

Создадим файл `src/contracts_impl/converter.rs` и объявим его в `src/contracts_impl/mod.rs`:

```
pub mod converter;
```

В `converter.rs` импортируем необходимые типы:

```
use rusqlite::{
    ToSql,
    types::{FromSql, FromSqlError, FromSqlResult, ToSqlOutput, Value,
ValueRef},
};
use time::{UtcDateTime, format_description::well_known::Rfc3339};

use crate::entities::{
    EntityId, ItemId, ItemQuantity, Name, PhoneNumber, Price, RecordLifetime,
};
```

Реализации SQL-трейтов остаются в слое адаптеров. Объекты-значения предоставляют метод `value()` для чтения внутреннего значения, но не позволяют менять закрытые поля в обход конструктора. Например, у `Price`:

```
impl Price {
    pub(crate) fn value(&self) -> f64 {
        self.0
```

```
    }
}
```

2. 2. Преобразуем объект в `Value`

Рассмотрим реализацию для цены:

```
impl From<Price> for Value {
    fn from(value: Price) -> Self {
        value.value().into()
    }
}
```

Внутри `Price` находится `f64`. Для него библиотека уже реализует преобразование в `Value::Real`, поэтому мы используем существующую реализацию.

После этого работает следующий код:

```
let price = Price::new(25.5)?;
let parameters: Vec<Value> = vec![price.into()];
```

2. 3. Передаём объект через `ToSql`

`ToSql` преобразует объект в представление для параметра SQL-запроса:

```
impl ToSql for Price {
    fn to_sql(&self) -> rusqlite::Result<ToSqlOutput<'_>> {
        Ok(ToSqlOutput::Owned(self.value().into()))
    }
}
```

```
impl ToSql for Name {
    fn to_sql(&self) -> rusqlite::Result<ToSqlOutput<'_>> {
        Ok(ToSqlOutput::Borrowed(ValueRef::Text(
            self.value().as_bytes(),
        )))
    }
}
```

Теперь объекты можно передавать в параметры непосредственно:

```
connection.execute(
    "UPDATE items SET price = ?1 WHERE id = ?2",
    rusqlite::params![&price, &item_id],
)?;
```

Здесь передаются ссылки: после выполнения запроса `price` и `item_id` остаются доступны. `ToSql` и `From` решают разные задачи; реализация одного трейта не создаёт автоматически реализацию другого.

2. 4. Читаем объект через `FromSql`

При чтении сначала получим базовый тип, затем вызовем проверяющий конструктор объекта:

```
impl FromSql for Price {
    fn column_result(value: ValueRef<'_>) -> FromSqlResult<Self> {
        Self::new(f64::column_result(value)?)
            .map_err(|error| FromSqlError::Other(Box::new(error)))
    }
}
```

Если значение имеет неподходящий SQL-тип, ошибку вернёт преобразование в `f64`. Если цена отрицательная, ошибку вернёт `Price::new`. Мы не создаём некорректный объект и не скрываем ошибку значением по умолчанию. Метод `row.get()` преобразует ошибку `FromSql` в `rusqlite::Error` с информацией о столбце.

Для `EntityId`, `ItemId`, `Name`, `PhoneNumber` и `ItemQuantity` используются такие же три реализации. Базовые представления различаются: идентификатор использует преобразования `Uuid`, имя и телефон — `String`, цена — `f64`, количество — `u32`.

Например, `ItemId` переиспользует преобразование `EntityId`:

```
impl FromSql for ItemId {
    fn column_result(value: ValueRef<'_>) -> FromSqlResult<Self> {
        Ok(Self::create(EntityId::column_result(value)?))
    }
}
```

После подключения конвертеров повторяющийся код восстановления товара сокращается:

```
let item_id: ItemId = row.get(0)?;
let name: Name = row.get(1)?;
let price: Price = row.get(2)?;
let lifetime = parse_record_lifetime(row, 3, 4, 5)?;

Ok(Item::restore(item_id, name, price, lifetime))
```

Примечание:

Проверки конструкторов сохраняются. При этом `Name::new` проверяет только свои правила; дополнительные ограничения конкретного сценария, например допустимую длину названия товара, нужно применять отдельно.

2. 5. Восстанавливаем составной объект

`RecordLifetime` отличается от скалярных объектов: он состоит из трёх дат и хранится в трёх столбцах. `FromSql` получает одно значение `ValueRef`, а `ToSql` возвращает один параметр. Поэтому эти трейты не могут сами распределить объект по нескольким столбцам.

Для существующей схемы используем отдельную функцию `parse_record_lifetime` из `converter.rs`:

```
pub fn parse_record_lifetime(
    row: &rusqlite::Row,
    created_at_index: usize,
    updated_at_index: usize,
    deleted_at_index: usize,
) -> Result<RecordLifetime, rusqlite::Error> {
    let created_at = parse_utc_datetime(row, created_at_index)?;
    let updated_at = parse_optional_utc_datetime(row, updated_at_index)?;
    let deleted_at = parse_optional_utc_datetime(row, deleted_at_index)?;

    RecordLifetime::create(created_at, updated_at, deleted_at).map_err(|error|
{
        rusqlite::Error::FromSqlConversionFailure(
            created_at_index,
            rusqlite::types::Type::Text,
            Box::new(error),
        )
    })
}
```

Индексы задают положение дат в конкретном `SELECT`. После разбора вызывается `RecordLifetime::create`, который проверяет взаимосвязь дат и допустимый диапазон. Ошибка конструктора сохраняется как причина ошибки преобразования.

Обязательная дата читается из строки в формате `RFC 3339`:

```
pub fn parse_utc_datetime(
    row: &rusqlite::Row,
    index: usize,
) -> Result<UtcDateTime, rusqlite::Error> {
    let text: String = row.get(index)?;
    UtcDateTime::parse(&text, &Rfc3339).map_err(|error| {
        rusqlite::Error::FromSqlConversionFailure(
            index,
            rusqlite::types::Type::Text,
            Box::new(error),
        )
    })
}
```

Для необязательной даты сначала проверяется SQL `NULL`. Непустое значение разбирается той же функцией `parse_utc_datetime`:

```
pub fn parse_optional_utc_datetime(
    row: &rusqlite::Row,
    index: usize,
) -> Result<Option<UtcDateTime>, rusqlite::Error> {
    if matches!(row.get_ref(index)?, ValueRef::Null) {
        return Ok(None);
    }
    parse_utc_datetime(row, index).map(Some)
}
```

Получаем три разных исхода: `NULL` становится `Ok(None)`, корректная дата — `Ok(Some(date))`, неправильная строка — `Err(...)`.

2. 6. Записываем составной объект

При записи `RecordLifetime` раскладывается на три значения. Используем тот же формат RFC 3339, который ожидает парсер:

```
let created_at = item.record_lifetime.created_at
    .format(&Rfc3339)
    .map_err(|_| DatabaseError::EntryMappingError)?;
let updated_at = item.record_lifetime.updated_at
    .map(|date| date.format(&Rfc3339))
    .transpose()
    .map_err(|_| DatabaseError::EntryMappingError)?;
let deleted_at = item.record_lifetime.deleted_at
    .map(|date| date.format(&Rfc3339))
    .transpose()
    .map_err(|_| DatabaseError::EntryMappingError)?;

let parameters: Vec<Value> = vec![
    item.id.into(),
    item.name.into(),
    item.price.into(),
    created_at.into(),
    updated_at.into(),
    deleted_at.into(),
];
```

Этот фрагмент применяется внутри метода с результатом `StorageResult`. `transpose()` преобразует `Option<Result<String, _>>` в `Result<Option<String>, _>`, позволяя обработать ошибку через `?`. Отсутствующие даты преобразуются в `Value::Null`.

Для реализации трёх SQL-трейтов у самого `RecordLifetime` пришлось бы выбрать представление в одном столбце, например JSON в `TEXT`, и изменить схему хранения. В текущей схеме сохраняем три столбца и используем функцию сборки объекта из строки.

2. 7. Что получаем в итоге

Преобразования сосредоточены в `converter.rs`, а запросы больше не дублируют создание объектов-значений и обработку их ошибок. `From` используется для `Vec<Value>`, `ToSql` — для параметров запроса, `FromSql` — для `row.get()`. Составные объекты восстанавливаются специальными функциями из нескольких столбцов.

Внутри SQL-замыкания ошибки остаются ошибками `rusqlite`; на границе адаптера они преобразуются в `DatabaseError`. Это отделяет формат хранения от бизнес-логики и позволяет переиспользовать одни и те же правила преобразования в разных хранилищах.

Задачи

> Выполняйте задания для объектов-значений своей предметной области. `ItemId`, `Name`, `Price` и `RecordLifetime` приведены как примеры: добавлять их в свой проект, если они ему не нужны, не требуется.

Задача 1. Определить SQL-представления объектов-значений

Найдите объекты-значения вашего проекта, которые используются при чтении и записи данных в `SQLite`. Определите, каким SQL-типом хранится каждый из них и какие объекты занимают несколько столбцов.

Задача 2. Создать модуль конвертеров

Создайте в слое адаптеров модуль `converter.rs` и подключите его. Для каждого объекта-значения, представленного одним SQL-значением, реализуйте `ToSql`, `FromSql` и `From<ОбъектЗначения> for Value` по примеру из методички. Для доступа к закрытым данным используйте методы чтения, не открывая поля для произвольного изменения.

Задача 3. Сохранить проверки конструкторов

При реализации `FromSql` используйте существующие проверяющие конструкторы объектов-значений. Ошибки преобразования и валидации должны возвращаться вызывающему коду, а не заменяться значениями по умолчанию.

Задача 4. Обновить адаптеры работы с БД

Обновите адаптеры работы с БД: используйте `row.get()` для чтения объектов-значений, `ToSql` для передачи параметров и `.into()` при формировании `Vec<Value>`. Уберите повторяющийся ручной код преобразования там, где его заменяют новые реализации.

Задача 5. Обработать составные объекты и необязательные поля

Если в вашей модели есть объекты-значения, хранящиеся в нескольких столбцах, создайте для них функции восстановления из строки БД по примеру `parse_record_lifetime`. При записи раскладывайте объект на отдельные параметры. Для необязательных полей сохраняйте соответствие `None` и SQL `NULL`, а для дат используйте одинаковый формат записи и чтения. Не объединяйте несколько столбцов в один только ради реализации SQL-трейтов.

Задача 6. Проверить преобразования

Проверьте запись и последующее чтение корректных значений, отказ при чтении данных, нарушающих правила конструктора, и обработку `NULL` для необязательных полей. Убедитесь, что восстановленные объекты сохраняют исходные значения. Исправьте ошибки компиляции, возникшие после подключения конвертеров.